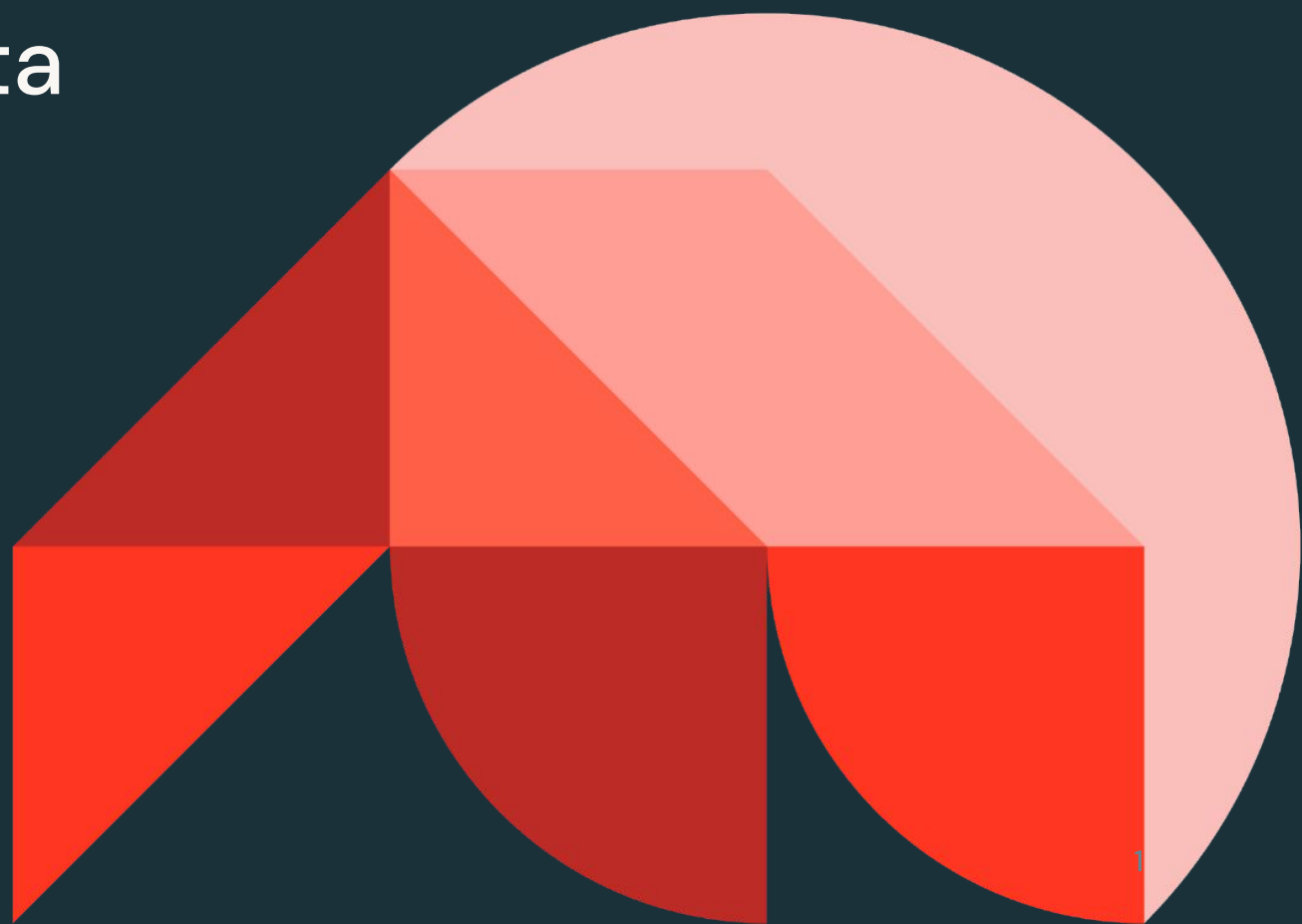




Data Model and Semantic Layer Architecture for Data Analytics– Briefing

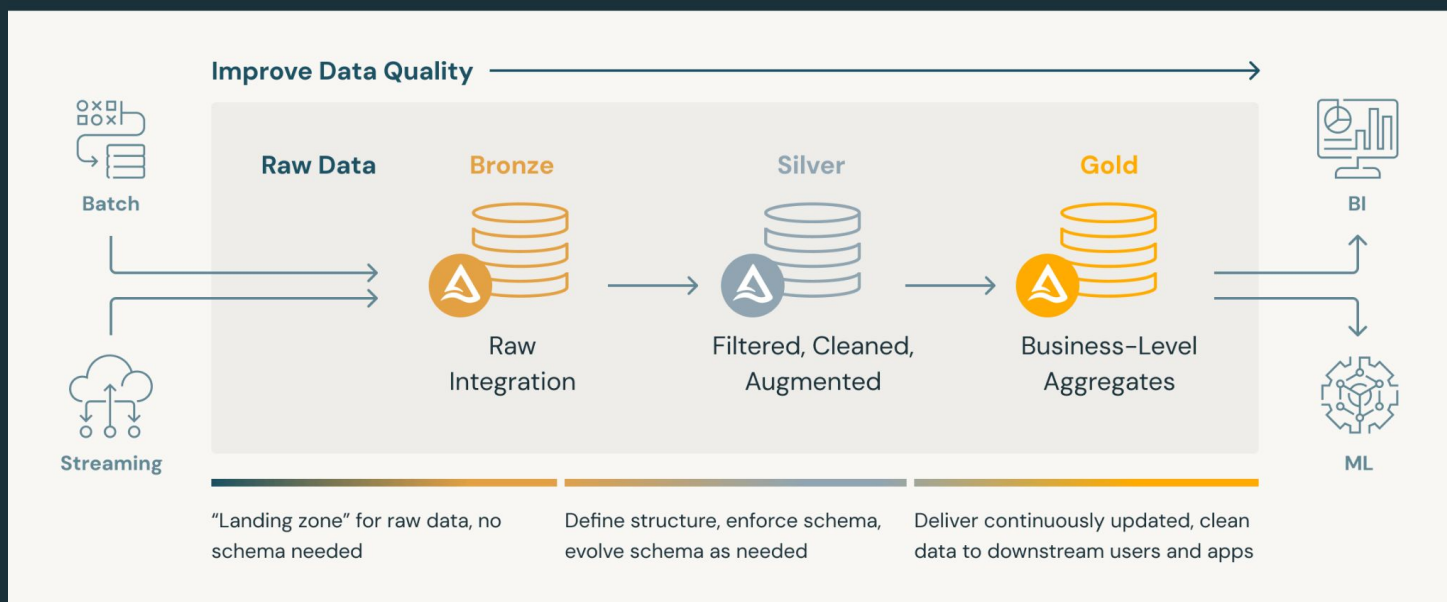
June 2025

©2024 Databricks Inc. — All rights reserved



Lakehouse Medallion Architecture

Databricks guidance for organizing data in a lakehouse



- **Bronze:** Landing and Ingestion
 - Data converted to Delta, no transformations applied
- **Silver:** Data Quality & Augment
 - Data is curated
 - **Semantically consistent**
- **Gold:** Presentation & Consumption
 - Consumer-orientated view
 - Cleansed, harmonised
 - Summaries and aggregations

UC Discovery Tags

Unity Catalog Semantic layer

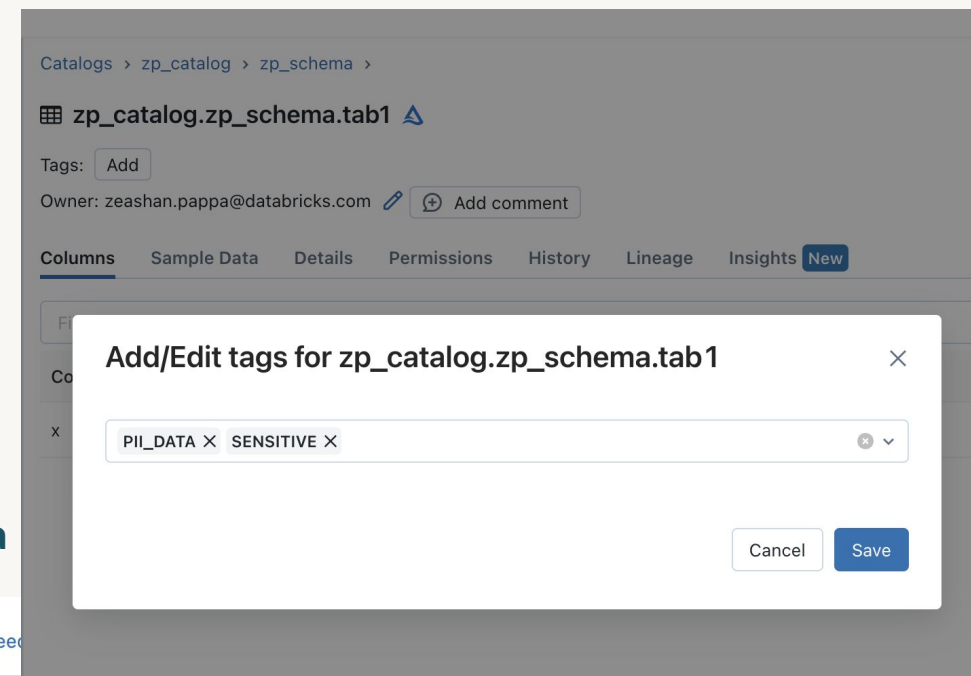
Problem

Searching for data assets in business terms or generally agreed upon taxonomies usually requires additional catalog tools.

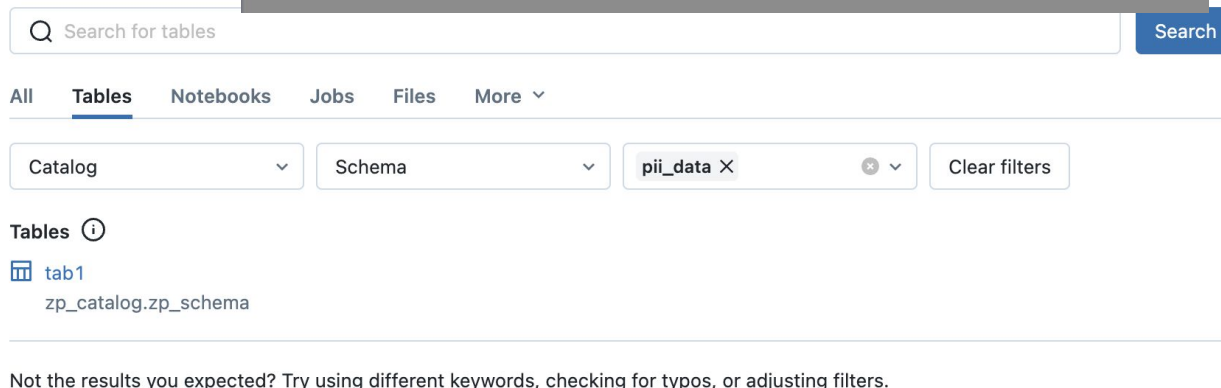
Solution

- Discovery Tags allow you to tag Column, Table, Schema, Catalog objects in UC
- **Integrated search mechanism in UC allows you to search for objects by tag.**

Tag your data



Search



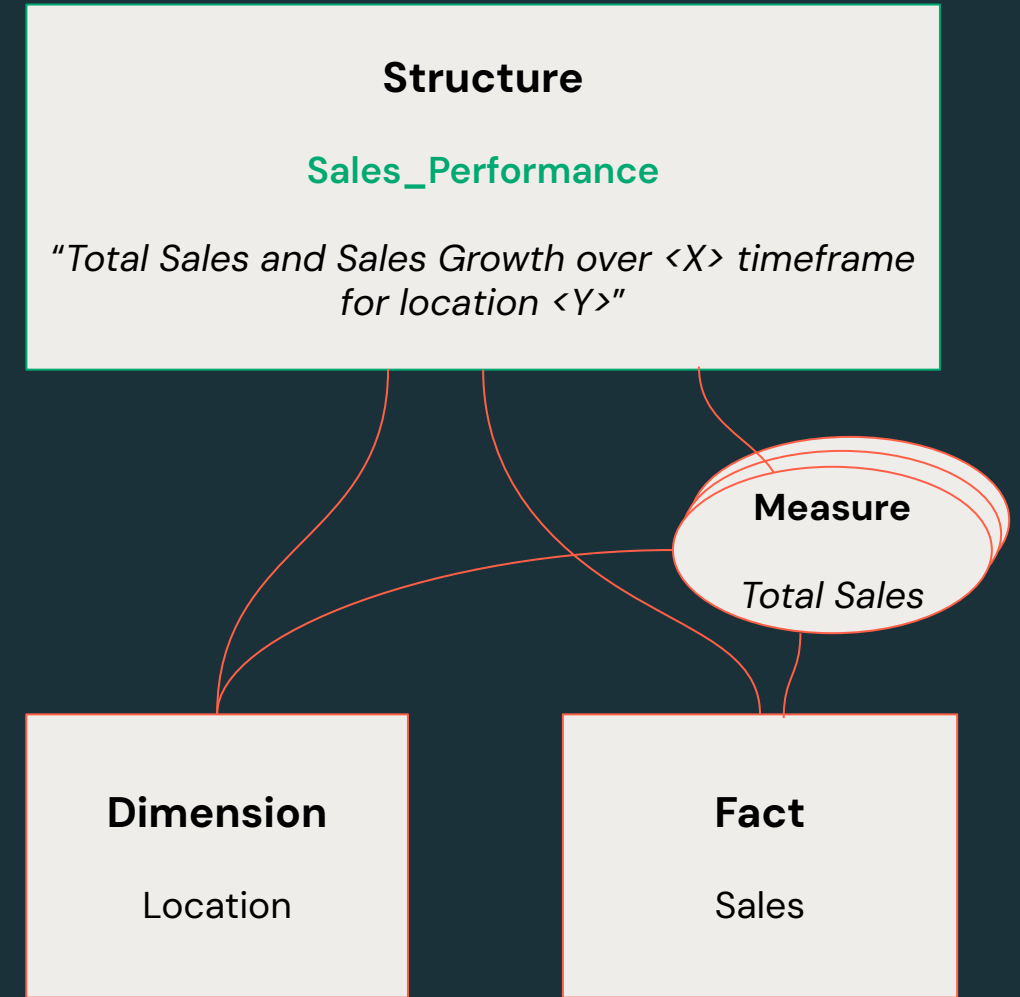
Search based on tags

Semantic Layer Concepts

Business-oriented data-model

Source of truth for Business Definitions

- **Facts** – measures or events
- **Dimensions** – dimensions or entities
- **Measures** – aggregations and calcs
- **Structures** – relationships and hierarchy
 - Facts, Dimensions, Calcs combined into a meaningful business concept
- **Governance and Metadata**
 - Consistent access controls and audit
 - Consistent labeling



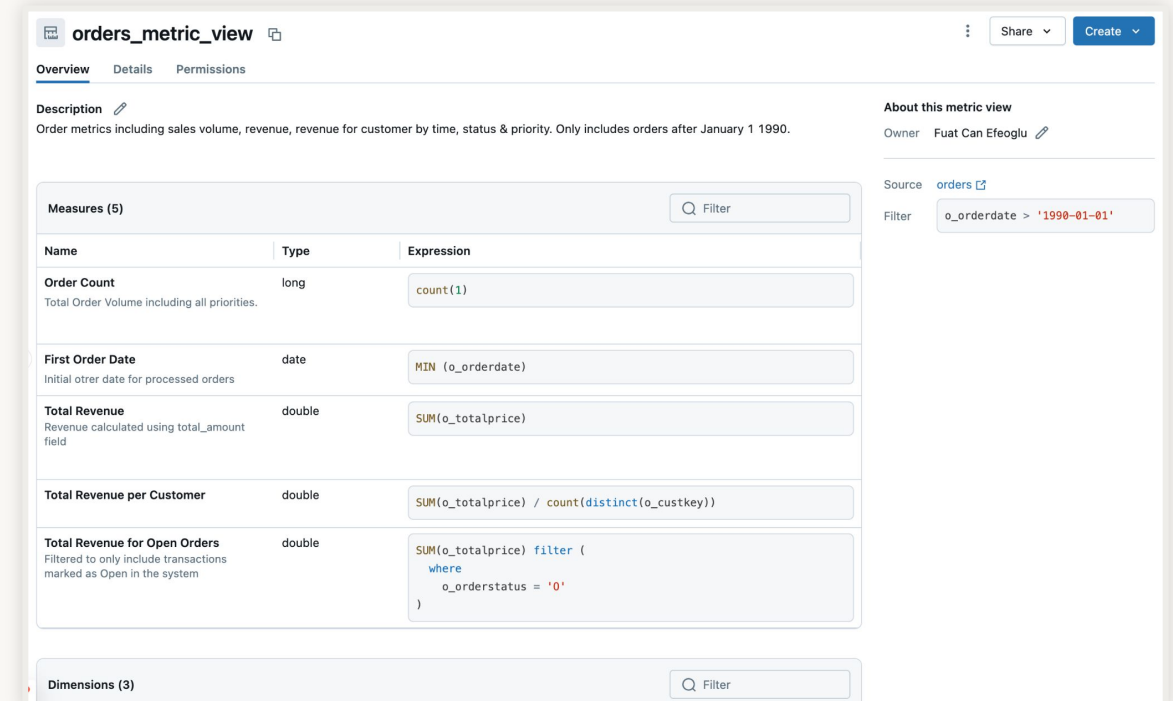
Reliable and consistent business metrics

Define metrics on top of your existing data

Centrally govern with search, usage insights, lineage & auditing built-in

Certify metrics for increased trust and reliability

Materialize for performance and pre-compute to reduce latency for frequent queries



The screenshot displays the 'orders_metric_view' interface. It includes a 'Description' section with a description of the metric view, an 'About this metric view' section with the owner's name, and a 'Measures (5)' table. The 'Measures' table lists five metrics: Order Count, First Order Date, Total Revenue, Total Revenue per Customer, and Total Revenue for Open Orders. Each measure includes its name, type, and the SQL expression used to calculate it. There is also a 'Dimensions (3)' section at the bottom.

Name	Type	Expression
Order Count Total Order Volume including all priorities.	long	count(1)
First Order Date Initial order date for processed orders	date	MIN(o_orderdate)
Total Revenue Revenue calculated using total_amount field	double	SUM(o_totalprice)
Total Revenue per Customer	double	SUM(o_totalprice) / count(distinct(o_custkey))
Total Revenue for Open Orders Filtered to only include transactions marked as Open in the system	double	SUM(o_totalprice) filter (where o_orderstatus = '0')



Defining a Metric View

YAML to define a Metric View

A metric view is made up of:

- Source: Table or view containing raw data
 - ✓ e.g., sales_transactions
- Dimensions: Fields to filter or group by
 - ✓ e.g., region, product, channel
- Measures: Aggregations or KPIs
 - ✓ e.g., total_sales, avg_discount, order_count

One metric view = Many questions answered

- Mix & match measures and dimensions
- Supports slicing and filtering without redefining
- No need to create new assets for every slice or filter

Catalog Explorer > can > metric_view_testing >

Update definition for orders_metric_view

```
1  version: 1.0
2
3  source: samples.tpch.orders
4  filter: o_orderdate > '1990-01-01'
5
6  dimensions:
7    - name: Order Month
8      expr: date_trunc('MONTH', o_orderdate)
9      type: date
10
11   - name: Order Status
12     expr: >
13       case
14         when o_orderstatus = 'O' then 'Open'
15         when o_orderstatus = 'P' then 'Processing'
16         when o_orderstatus = 'F' then 'Fulfilled'
17       end
18     type: string
19
20   - name: Order Priority
21     expr: split(o_orderpriority, '-')[1]
22     type: string
23
24  measures:
25    - name: Order Count
26      expr: count(1)
27      type: bigint
28
29    - name: First Order Date
30      expr: MIN(o_orderdate)
31      type: date
32
33    - name: Total Revenue
34      expr: SUM(o_totalprice)
35      type: double
36
37    - name: Total Revenue per Customer
38      expr: SUM(o_totalprice) / count(distinct(o_custkey))
39      type: double
40
```



Simplified Consistent Insights

Business question

"What's the average revenue per customer per segment for fulfilled orders since Jan 1, 2024?"

Equivalent query

```
SELECT
  c.customer_segment,
  SUM(o.revenue) / COUNT(DISTINCT
    c.customer_id) AS revenue_per_customer
FROM orders o
JOIN customer c ON o.c_id=c.id
WHERE order_date > '2024-01-01' AND
o.status = 'fulfilled' AND o.check IS
NOT NULL
GROUP BY c.customer_segment;
```

- ✗ Repeated logic
- ✗ Easy to make mistakes
- ⚠ Need to understand underlying tables

UC Metric View Query

```
SELECT
  'Customer Segment',
  MEASURE('Revenue per Customer')
FROM customer_metrics
WHERE 'Order Category' = "Valid
Fulfilled"
GROUP BY ALL
```

- ✓ Simpler queries
- ✓ Defined once, reused everywhere
- ✓ Fully governed & centralized



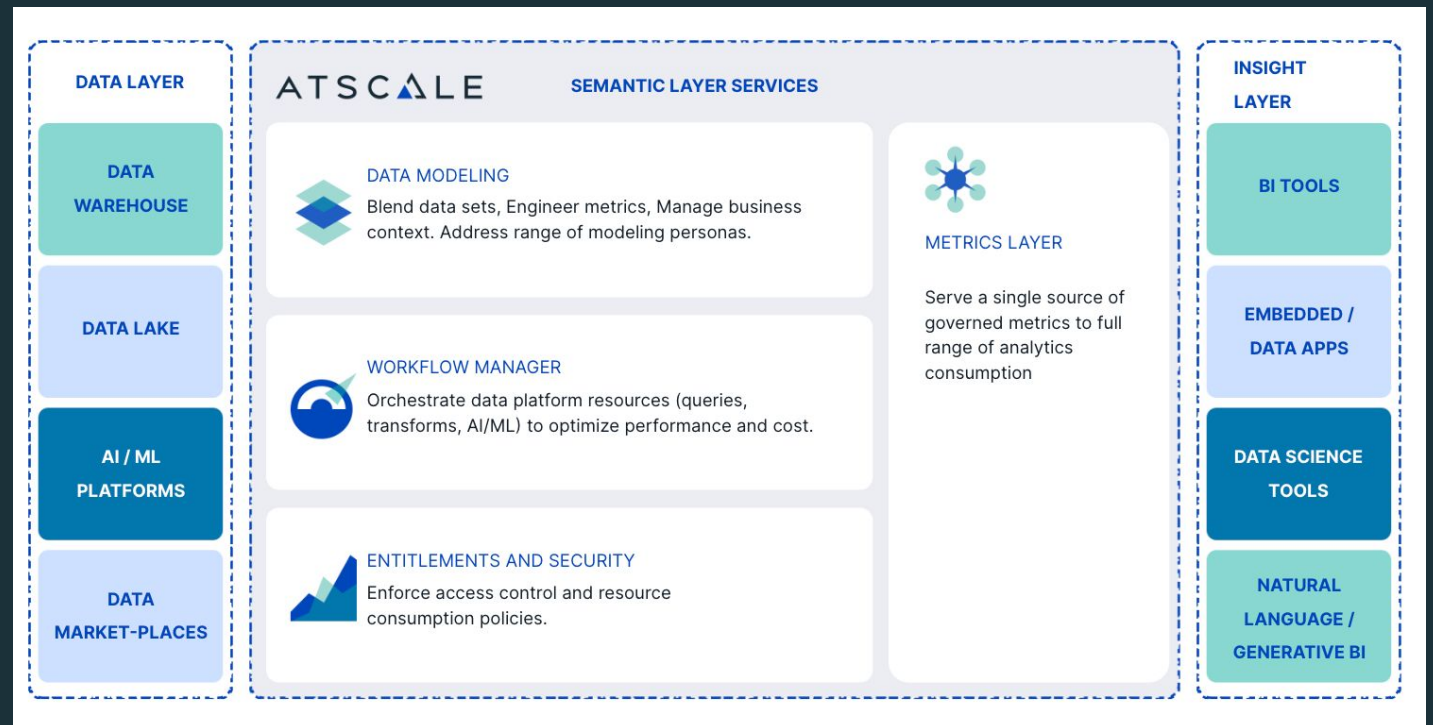
Partner Products and Alternatives



AtScale product view

Third party vendor solution

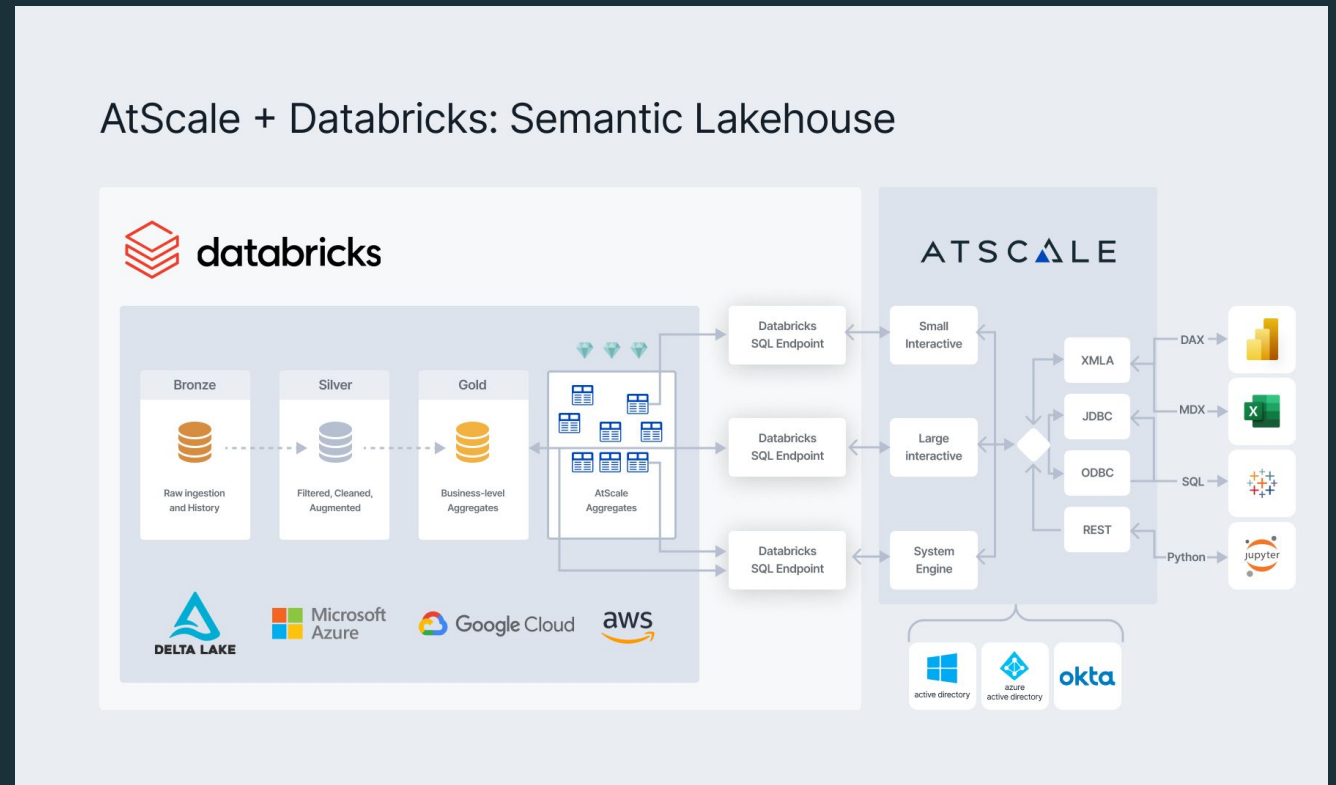
- **Semantic Model in BI Tool**
Traditionally, semantic models were built by dashboard creators within BI tools
→ *workgroup / team sprawl*
- **Semantic Model in Data Warehouse**
Data assets in the DW are the single source of truth
→ *Need a business-ready view, else leads to local workgroup models and sprawl*
- **Semantic Layer in Data Pipelines**
Embedding semantic layer logic into the code that defines pipelines
→ *hard to manage and ensure consistency as pipelines evolve and scale*
- **Universal Semantic Layer**
independent layer between raw data assets and data consumers
→ *decoupled solution abstracts complexity and avoids duplication.*



AtScale and Databricks integration

Databricks and AtScale partnership

- **Universal Semantic Layer**
AtScale delivers a business-oriented semantic layer that sits on top of the Databricks Lakehouse
- **BI Tools integration**
Users to run low latency BI directly on the Delta Lake
- **Workload push-down**
AtScale drives analytics workloads to Databricks SQL endpoints and clusters to deliver sub second query performance
- **Aggregates and Metrics**
Automates the creation and maintenance of aggregate tables in Delta Lake



DBT Mesh and MetricFlow

Databricks and DBT partnership

DBT Mesh

- DBT Mesh is not a semantic layer, instead it is an architectural pattern within the dbt ecosystem
- DBT Mesh facilitates scalable, decentralized data collaboration across multiple teams and projects
- It complements the dbt Semantic Layer, which focuses on centralizing and standardizing metric definitions

DBT Mesh Features

- **Cross-Project References:** models in one dbt project reference models in another
- **Model Contracts and Versions:** enforce data structure standards and manage changes over time
- **Access Control:** define who can access or modify models
- **Lineage:** visibility into data flow across projects

DBT MetricFlow (Semantic Layer)

- MetricFlow was acquired from Transform in 2023
- Metric YAML configurations are defined on top of DBT models
- Entities, Dimensions and Measures are defined to provide business-functional insights on data

DBT MetricFlow Features

- **Define Metrics Once:** standardized metric definitions in YAML files within dbt projects
- **Integrate with BI Tools:** expose metrics to downstream tools like Tableau, Power BI
- **Maintain Consistency:** ensure that all teams use the same definitions, reducing discrepancies in reporting.
- **Dynamic SQL Generation:** Automatically generate optimized SQL queries for metrics

Power BI Semantic Layer

Move semantic layer to the BI Tool

Power BI Semantic Model

- Formerly known as a Power BI **dataset**
- Collection of tables, relationships, and calculations *defined within Power BI*
- Access data using DirectQuery or Import Mode.
- Store data in memory for optimized performance, and larger models can be enabled with Premium capacities
- Define measures and KPIs using **DAX** (Data Analysis Expressions)
- Implement row-level security (RLS) within the Power BI dataset to control data access per user/team.

Power BI Features

- **Centralized Logic:** Define metrics (e.g. revenue, churn) once in DAX; reused across reports and teams.
- **Self-Service BI:** Empower analysts and business users to build consistent reports without duplicating metric logic.
- **Performance Optimization:** built-in aggregation tables, hybrid models (import + DirectQuery), and MVs.

Design Considerations

- **Power BI Specific:** strategy based on Power BI specific code and functionality (DAX etc)
- **Move the security model to BI tool:** RLS, measure entitlements etc
- **Move the data access layer to BI tool:** business-level view of data has moved out of the data platform.

Summary



Semantic Layer for Databricks Lakehouse

Options and Decision Points

Native Databricks Strategic Solution

Integrate the semantic layer into the base data model and security model.
Serve to multiple consumer groups and consumer tools.

Universal Semantic Layer

Apply the semantic layer across multiple data platforms. Security and governance at the semantic layer moves to the semantic layer tool.

ELT Tool Semantic Layer

Define the semantic layer in the data transformation tool. Security and governance at the semantic layer moves to the semantic layer tool.

BI Semantic Layer

Define the semantic layer in the BI Tool. Definitions in BI-tool specific language. Access point (and security model) all via the BI Tool.

Prioritisation

- Medallion Architecture + Business Level Aggregates
 - Unity Catalog and Discovery Tags
 - Semantic Layer Business Metrics

